Software Requirements Specification

Daily Standup Summarizer

Version: 1.0 **Date:** 16 June 2026 **Status:** Draft

---

1. Introduction

1.1 Purpose

This document specifies the requirements for the Daily Standup Summarizer, an automated pipeline that collects daily status messages posted by team members in a Slack channel, generates a concise per-person summary using a language model, and records those summaries in a Google Sheet for review.

1.2 Scope

The system is a scheduled, non-interactive Python application. Once per day it pulls the day's messages from a designated Slack channel, groups them by author, produces a structured summary for each person, and appends or updates rows in a Google Sheet. It runs unattended via a scheduler (e.g. cron) and requires no human intervention in the normal path.

Out of scope for version 1.0: a user interface, real-time/streaming processing, two-way Slack interaction (e.g. replying to people), and validation of reported work against an external task tracker. The last item is identified as a planned future extension (see Section 11).

1.3 Definitions and acronyms

Term	Meaning
Standup	A team member's daily written status update posted in Slack
Reasoning engine	The language model that converts raw messages into structured summaries
Hosted backend	A pay-per-token model accessed over the network (e.g. Anthropic Claude API, Alibaba DashScope)
Local backend	An open-weight model run on owned hardware (e.g. Qwen3 via Ollama)
Run window	The time range whose messages count as "today's standup"
Upsert	Insert a new row, or overwrite the existing row for the same person and date
SRS	Software Requirements Specification

1.4 Overview

Section 2 describes the system at a high level. Section 3 defines the architecture and modules. Sections 4–7 specify functional, interface, data, and non-functional requirements. Section 8 details the pluggable reasoning engine and the hosted-vs-local decision. Sections 9–11 cover configuration, acceptance, and future work.

---

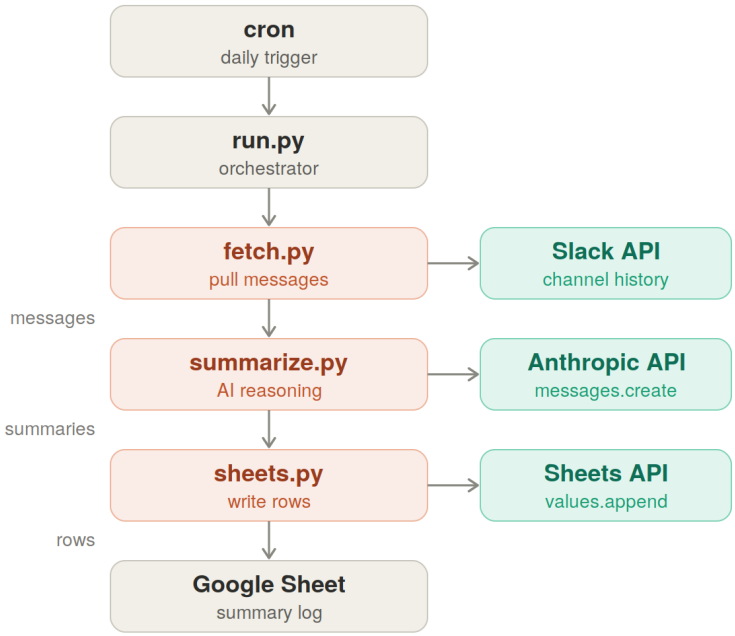
2. Overall description

2.1 Product perspective

The product is a linear, single-direction data pipeline. A scheduler triggers an orchestrator, which calls three modules in sequence. Each module owns exactly one external service, and the only thing passed between stages is data:

```
cron --> run.py --> fetch.py --> summarize.py --> sheets.py --> Google Sheet
                       |             |               |
                  Slack API   Reasoning API     Sheets API
```

The model is invoked exactly once per run, on the whole team's messages in a single batched call. This keeps cost predictable and low; everything on either side of the model is ordinary HTTP and consumes no tokens.



One batched model call per run. Slack and Sheets are plain HTTP (no token cost).

Figure 1 — End-to-end pipeline. cron triggers run.py, which calls fetch, summarize and push in order; each module owns one external service, and the model is invoked exactly once per run.

2.2 Product functions

At a high level the system shall: 1. Retrieve the day's messages from a configured Slack channel within the run window. 2. Resolve author identities and group messages by person. 3. Generate a structured summary per person (summary text, status, blockers). 4. Detect team members who posted no update. 5. Write the results to a Google Sheet, avoiding duplicate rows on re-runs. 6. Run on a daily schedule without manual intervention.

2.3 User characteristics

The primary user is a team lead or manager who reads the Google Sheet. The operator who installs and configures the system is a developer comfortable with Python, environment variables, and basic cloud/API credential setup. End users (team members) interact only with Slack as they already do; they are not required to change their posting habits, though an optional template improves summary quality.

2.4 Constraints

- The system is implemented in Python 3.10+ for ease of extension and testing.
- It must not require an always-on server; a scheduled process is sufficient.
- Token cost must remain bounded to a single model call per run.
- The reasoning engine must be replaceable without changes to other modules.

2.5 Assumptions and dependencies

- A Slack workspace exists with a bot token that has read access to the target channel.
- A Google Cloud service account exists and the target sheet is shared with it.
- A reasoning backend is available — either network access to a hosted API, or local hardware running an open-weight model.
- `jq`-style parsing is handled in Python; official SDKs are used where available (`slack_sdk`, `gspread` / `google-auth`, and the chosen model's client).
- The host running the scheduler has a stable clock and correct timezone configuration.

3. System architecture

The application is a small Python package with one responsibility per module:

Module	Responsibility	External service
`config.py`	Load settings and secrets from environment; expose typed config	—
`fetch.py`	Pull channel history for the run window; resolve user IDs to names; group by person	Slack API
`summarize.py`	Transform grouped messages into structured summaries via the reasoning engine	Reasoning API (pluggable)
`sheets.py`	Read existing rows; upsert summary rows	Google Sheets API
`run.py`	Orchestrate the pipeline; compute the run window; handle errors and logging; serve as cron entry point	—

The summary data shape is defined once (a dataclass or Pydantic model) and flows through `summarize.py` and `sheets.py`, so adding a field is a localized change.

---

4. Functional requirements

4.1 Fetch (`fetch.py`)

- **FR-1.** The system shall retrieve all messages posted to the configured Slack channel within the run window, handling pagination.
- **FR-2.** The system shall resolve Slack user IDs to display/real names so output is keyed by person, not by raw ID.
- **FR-3.** The system shall group messages by author, preserving message order and timestamps.
- **FR-4.** The system shall include or exclude thread replies according to a configuration flag (default: include).
- **FR-5.** The system shall exclude bot and system messages from the grouped output.
- **FR-6.** If the channel returns no messages for the window, the system shall proceed with an empty set rather than failing.

4.2 Summarize (`summarize.py`)

- **FR-7.** The system shall send the grouped messages to the reasoning engine in a single batched request per run (not one request per person).
- **FR-8.** For each person, the system shall produce a structured record containing at minimum: person, summary, status, blockers.
- **FR-9.** The system shall instruct the engine to return machine-parseable output (JSON) and shall validate it against the defined schema.
- **FR-10.** If a person's messages are ambiguous or empty, the system shall still emit a record with an appropriate status (e.g. `no-update`).
- **FR-11.** The reasoning engine shall be selectable by configuration between supported backends without code changes in other modules (see Section 8).
- **FR-12.** If the engine returns malformed output, the system shall retry up to a configurable number of attempts before recording an error status for the affected run.

4.3 Push (`sheets.py`)

- **FR-13.** The system shall write one row per person per day to the configured Google Sheet.
- **FR-14.** The system shall read existing rows for the run's date and upsert: overwrite a matching person+date row, or append if none exists, so re-running a day does not create duplicates.
- **FR-15.** The system shall flag team members who were expected but posted no update, where an expected-members list is configured.
- **FR-16.** Date and status values shall be written so the sheet interprets them naturally (using the appropriate value-input option).

4.4 Orchestration and scheduling (`run.py`)

- **FR-17.** The system shall compute the run window from the configured timezone and window definition.

- **FR-18.** The system shall execute fetch, summarize, and push in order, halting the run and logging a clear error if any stage fails irrecoverably.
- **FR-19.** The system shall be runnable both on a schedule (cron) and manually for a specified date, to support backfilling and testing.
- **FR-20.** The system shall produce structured logs sufficient to diagnose a failed run without exposing secrets.

---

5. External interface requirements

5.1 Slack API

- Uses the channel-history endpoint bounded by start/end timestamps, plus user-lookup for ID resolution.
- Requires a bot token with `channels:history` (and `groups:history` for private channels) and `users:read`.
- The bot must be a member of the target channel.

5.2 Reasoning engine API

- Abstracted behind a single internal interface (e.g. `summarize(messages) -> list[Summary]`).
- Concrete backends:
- **Hosted:** an HTTP API reached with an API key (e.g. Anthropic Messages API, or Alibaba DashScope for hosted Qwen).
- **Local:** an OpenAI-compatible endpoint exposed by Ollama or vLLM serving an open-weight model such as Qwen3, reached at a configurable base URL with no per-token cost.
- The interface contract is identical across backends; only the client and endpoint differ.

5.3 Google Sheets API

- Uses Sheets API v4 values endpoints (`get` for existing rows, `append` / `update` for writes).
- Authenticated with a service account; the target spreadsheet must be shared with the service account's email as Editor.
- The spreadsheet is identified by its sheet ID.

---

6. Data requirements

6.1 Grouped messages (fetch output)

A mapping from person to an ordered list of `{timestamp, text}` entries, plus the resolved display name.

6.2 Summary record (summarize output / sheet row)

Field	Type	Description
date	date	The standup date (run window date)
person	string	Resolved member name
summary	string	Concise account of the person's reported work
status	enum	e.g. `on-track` , `blocked` , `no-update`
blockers	string	Reported blockers, or `none`
posted	boolean	Whether the person posted at all that day

The record is defined as a single typed model and reused across modules; adding a column (e.g. `tickets_closed`) is a one-field change.

6.3 Google Sheet layout

A single master-log tab, one row per person per day, with columns matching the summary record. A derived per-person view may be layered on later without changing the source of truth.

---

7. Non-functional requirements

7.1 Cost

- **NFR-1.** Each run shall make exactly one reasoning-engine call. With a local backend the marginal token cost is zero; with a hosted backend the cost is bounded by the day's message volume plus the summaries.

7.2 Performance

- **NFR-2.** A normal daily run (a single team, tens of messages) shall complete within a few minutes, dominated by the model call.

7.3 Reliability

- **NFR-3.** A failure in any single stage shall not corrupt the sheet; partial writes shall be avoidable via upsert semantics and clear failure logging.
- **NFR-4.** Re-running the same date shall be idempotent (no duplicate rows).

7.4 Security and privacy

- **NFR-5.** All credentials (Slack token, service-account key, model API key) shall be supplied via environment or secret files, never hard-coded.
- **NFR-6.** No secrets shall appear in logs.
- **NFR-7.** Where status data is sensitive, the local reasoning backend shall be available so message content never leaves owned infrastructure.

7.5 Maintainability and extensibility

- **NFR-8.** Each module shall be independently unit-testable against fixtures without live API calls.
- **NFR-9.** The reasoning engine shall be swappable by configuration alone.
- **NFR-10.** The summary schema shall be defined in one place.

---

8. Reasoning engine: hosted vs. local (Qwen)

A central design decision is whether the reasoning engine is a paid hosted model or a free local open-weight model. The architecture supports both behind one interface, so the choice is configuration, not code.

8.1 Why a local model is viable

Standup summarization is a bounded transformation: group short messages, write a few sentences per person, classify a status. This is well within the capability of small open-weight models. Open-weight Qwen3 models (Apache 2.0) run locally through Ollama or vLLM, which expose an OpenAI-compatible endpoint, so the same client code targets either a hosted API or `localhost` by changing the base URL.

8.2 Suggested local options

Model	Footprint	Suitability
Qwen3-8B / 14B	Single modest GPU or strong laptop	Comfortable for grouping and per-person summaries
Qwen3-30B-A3B (MoE)	~3B active params	Stronger quality, still efficient — recommended sweet spot
Larger / hosted Qwen (DashScope)	No local hardware	If sharper "check the work" judgment is wanted without managing GPUs

8.3 Trade-offs

Dimension	Hosted (paid)	Local (Qwen)
Marginal cost	Pay per token	Zero
Data residency	Leaves your network	Stays on owned hardware
Operational burden	None	You own hardware + uptime
Quality ceiling	Frontier	High; sufficient for this task
Setup	API key only	Install Ollama/vLLM, pull model

8.4 Requirement

- **FR-11 (restated).** The system shall select the reasoning backend via configuration. Supported backends in v1.0: a hosted API and a local OpenAI-compatible endpoint serving an open-weight Qwen3 model. Switching backends shall not require changes to `fetch.py`, `sheets.py`, or `run.py`.

---

9. Configuration

All of the following shall be supplied via environment variables or a config file, with no secrets in source control: - Slack: bot token, channel ID, include-threads flag. - Reasoning engine: backend selector (`hosted` | `local`), model name, base URL (for local), API key (for hosted). - Google Sheets: service-account key path, spreadsheet ID, tab name. - Schedule: run window definition and timezone, expected-members list (optional).

---

10. Acceptance criteria and phased delivery

The system is built and validated in phases to de-risk the hard parts first:

1. **Phase 1 — Summarize.** `summarize.py` plus the summary schema produce valid structured output from pasted sample messages. Acceptance: given fixture messages, the module returns schema-valid records for each person, including a `no-update` case. The reasoning backend is swappable between hosted and local at this stage.
2. **Phase 2 — Fetch.** `fetch.py` returns correctly grouped, name-resolved messages for the run window from the live channel.
3. **Phase 3 — Push.** `sheets.py` writes rows and upserts correctly; re-running a date produces no duplicates.
4. **Phase 4 — Schedule.** `run.py` ties the stages together and runs unattended on cron, with logging and error handling, plus a manual backfill mode.

---

11. Future extensions

- **Work validation ("check the work").** A `tasks.py` module pulls each person's assigned items from a task tracker (e.g. Jira, Asana, Linear) and supplies them to `summarize.py` as additional context, enabling the engine to flag mismatches between reported and assigned work. This slots in between fetch and summarize without changing the existing modules.
- **Per-person dashboard tab** derived from the master log.
- **Notifications** (e.g. a Slack message back to the lead) once summaries are written.
- **Trend reporting** across days/weeks from the accumulated log.

End of document.